



**Universidad Nacional
Autónoma de México**
Facultad de Ciencias
Fundamentos de Bases de Datos



Bitácora: Instalación de Docker, PostgreSQL y DBeaver

Practica01

Profesor: Gerardo Avilés Rosas

Ayudante de teoría: Oscar Daniel Flores Linares

Ayudante de laboratorio: Ricardo Badillo Macías

Núñez Ruíz José Manuel

No. de Cuenta: 322112072

Fecha de entrega: 7 de septiembre de 2026

1. Sistema operativo y versión:

Arch Linux con kernel: 7.2.3-arch1-2.

2. Distribución:

Arch Linux x86_64.

3. Versión de la instalación:

- **Docker:** Docker version 29.7.2, build a7dcaa6fdb.
- **PostgreSQL:** psql (18.6 (Debian 18.6-1.pgdg13+2))
- **DBeaver:** DBeaver 26.2.0.202608302102.

4. Tiempo requerido:

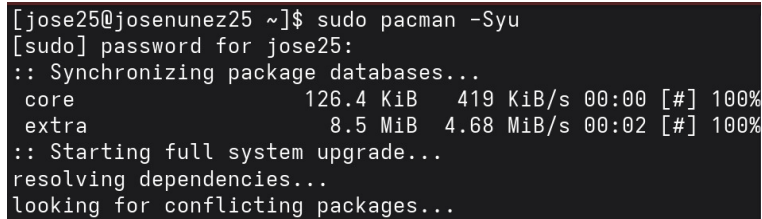
1 hora y 20 minutos.

5. Explicación paso a paso:

Instalación de Docker

1. Primero actualizamos los paquetes del sistema con el siguiente comando:

```
sudo pacman -Syu
```

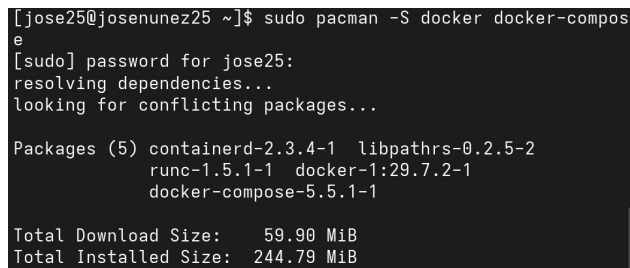


```
[jose25@josenuñez25 ~]$ sudo pacman -Syu
[sudo] password for jose25:
:: Synchronizing package databases...
core               126.4 KiB   419 KiB/s 00:00 [#] 100%
extra              8.5 MiB   4.68 MiB/s 00:02 [#] 100%
:: Starting full system upgrade...
resolving dependencies...
looking for conflicting packages...
```

Figura 1: Actualización del sistema

En Arch Linux la instalación me pareció más sencilla (Con la ayuda de la Arch wiki [1]), ya que no se requieren agregar repositorios externos de forma manual; Docker está disponible directamente en los repositorios oficiales. Además, leí la recomendación de instalar también el plugin de Docker Compose, ya que está empaquetado por separado. Por ello, se utilizó:

```
sudo pacman -S docker docker-compose
```



```
[jose25@josenuñez25 ~]$ sudo pacman -S docker docker-compos
e
[sudo] password for jose25:
resolving dependencies...
looking for conflicting packages...

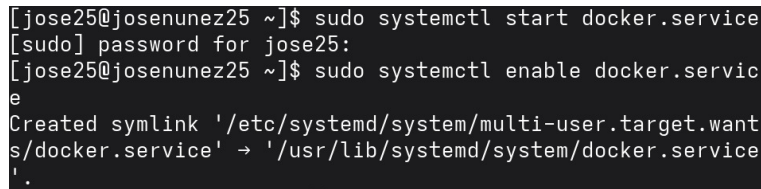
Packages (5) containerd-2.3.4-1 libpathrs-0.2.5-2
runc-1.5.1-1 docker-1:29.7.2-1
docker-compose-5.5.1-1

Total Download Size:   59.90 MiB
Total Installed Size: 244.79 MiB
```

Figura 2: Instalación de Docker y dependencias

- Después procedemos a inicializar y habilitar el servicio, pues en Arch Linux los servicios nuevos no se inician de manera automática:

```
sudo systemctl start docker.service
sudo systemctl enable docker.service
```

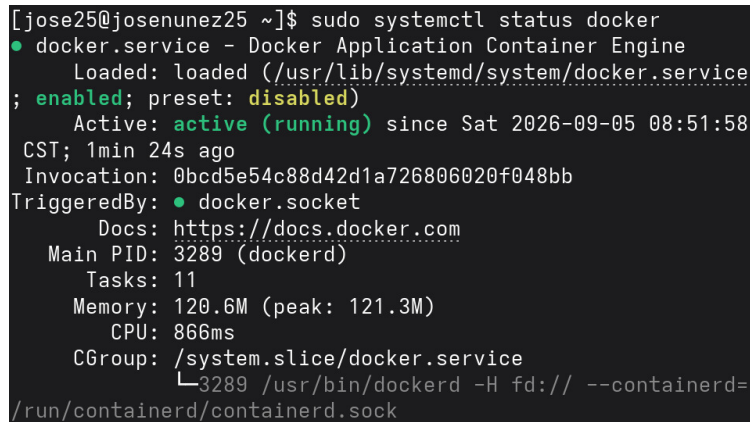


```
[jose25@josenunez25 ~]$ sudo systemctl start docker.service
[sudo] password for jose25:
[jose25@josenunez25 ~]$ sudo systemctl enable docker.servic
e
Created symlink '/etc/systemd/system/multi-user.target.want
s/docker.service' -> '/usr/lib/systemd/system/docker.service
'.
```

Figura 3: Inicialización y habilitación de Docker

- Comprobamos el servicio con el comando:

```
sudo systemctl status docker
```



```
[jose25@josenunez25 ~]$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service
; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-05 08:51:58
CST; 1min 24s ago
   Invocation: 0bcd5e54c88d42d1a726806020f048bb
   TriggeredBy: ● docker.socket
      Docs: https://docs.docker.com
    Main PID: 3289 (dockerd)
       Tasks: 11
    Memory: 120.6M (peak: 121.3M)
        CPU: 866ms
    CGroup: /system.slice/docker.service
            └─3289 /usr/bin/dockerd -H fd:// --containerd=
/run/containerd/containerd.sock
```

Figura 4: Estado del servicio de Docker

- Probamos si la instalación fue exitosa:

```
sudo docker run hello-world
```

```
^C[jose25@josenunez25 ~]$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Figura 5: Prueba del contenedor Hello World

Creación del contenedor de PostgreSQL

1. Descargar la imagen de PostgreSQL:

```
sudo docker pull postgres
```

```
[jose25@josenunez25 ~]$ sudo docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
1bb54e8f7727: Pull complete
af2a6480a058: Pull complete
5ee6c6ca7fd9: Pull complete
aedab7177504: Pull complete
a4635eaec9b8: Pull complete
ace4b5b96a91: Pull complete
5fa29c524b4e: Pull complete
46e663abc8f5: Pull complete
6310eb16bf42: Pull complete
6463c6b5c703: Pull complete
8d6422da287d: Pull complete
2fc0e69f8d7d: Pull complete
c5694bfff1357: Pull complete
6ff88415323f: Download complete
041cb76c0e1d: Download complete
Digest: sha256:4ef4dbc939d61acea57712655ddb4b4ab27419c913f94cca0cd57cb3ea3c2280
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
```

Figura 6: Descarga de la imagen de PostgreSQL

2. Crear el contenedor en la terminal:

```
sudo docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=***** \
  -p 127.0.0.1:5432:5432 \
  -v postgres_data:/var/lib/postgresql \
  postgres
```

3. Obtenemos su ID y probamos detenerlo e iniciarlo con:

```
sudo docker ps -a
```

```
[jose25@josenunez25 ~]$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
fd7860390858	postgres	"docker-entrypoint.s..."	About a minute ago	Up About a minute	127.0.0.1:5432->
5432/tcp	postgres				
bc7f05a6f1ea	hello-world	"/hello"	12 minutes ago	Exited (0) 12 minutes ago	
determined_burnell					

Figura 7: Visualización del ID del contenedor

Después lo detenemos y lo inicializamos con:

```
sudo docker stop [id_container]
sudo docker start [id_container]
```

```
[jose25@josenunez25 ~]$ sudo docker stop fd7860390858
fd7860390858
[jose25@josenunez25 ~]$ sudo docker start fd7860390858
fd7860390858
```

Figura 8: Parar e iniciar el contenedor PostgreSQL

4. Finalmente, si nos queremos conectar a PostgreSQL una vez creado el contenedor, corremos:

```
sudo docker run -it --link postgres:postgres postgres psql -h postgres -U postgres
```

```
[jose25@josenunez25 ~]$ sudo docker run -it --link postgres:postgres postgres psql -h postgres -U postgres
WARNING: Links on the default bridge network are deprecated and will be removed in a future release. Use a custom network instead.
Password for user postgres:
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

postgres=# \q
```

Figura 9: Conexión a PostgreSQL por terminal

Instalación y conexión a DBeaver

Encontré más conveniente instalarlo desde la terminal.

1. Instalación con Pacman:

```
sudo pacman -S dbeaver
```

```
[jose25@josenunez25 ~]$ sudo pacman -S dbeaver
[sudo] password for jose25:
Sorry, try again.
[sudo] password for jose25:
Sorry, try again.
[sudo] password for jose25:
resolving dependencies...
looking for conflicting packages...

Packages (1) dbeaver-26.2.0-1

Total Download Size:    99.53 MiB
Total Installed Size:  112.80 MiB
```

Figura 10: Instalación de DBeaver

2. Configuramos DBeaver:

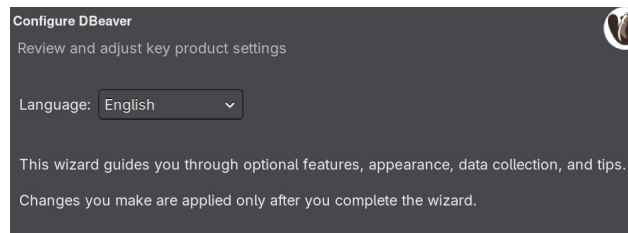


Figura 11: Configuración inicial en DBeaver

3. Después me conecté a DBeaver con los siguientes parámetros:

- **Host:** localhost o 127.0.0.1
- **Port:** 5432
- **Username:** postgres
- **Password:** ***** (oculta por seguridad)

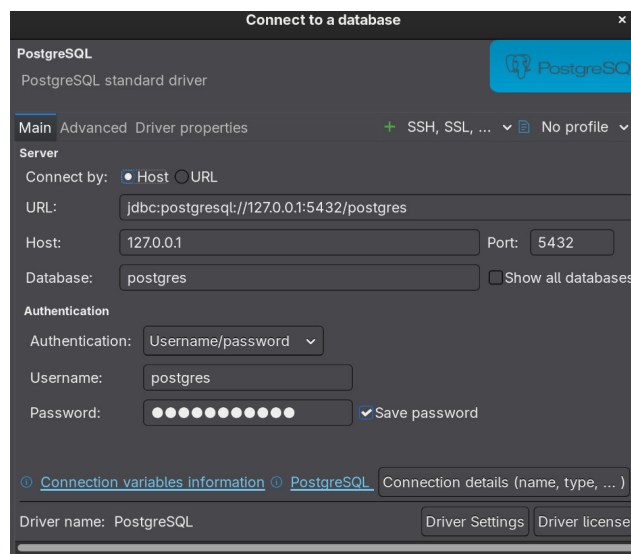


Figura 12: Parámetros de conexión

4. Nos pide descargar algunos drivers:

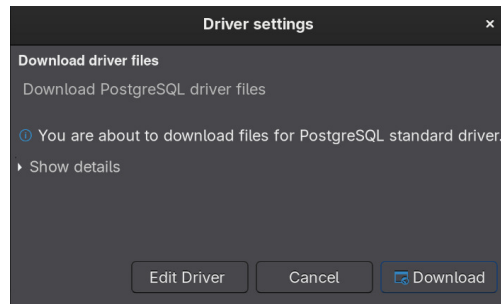


Figura 13: Descarga de drivers de PostgreSQL

5. Finalmente, la conexión es exitosa como se puede ver en la siguiente imagen:

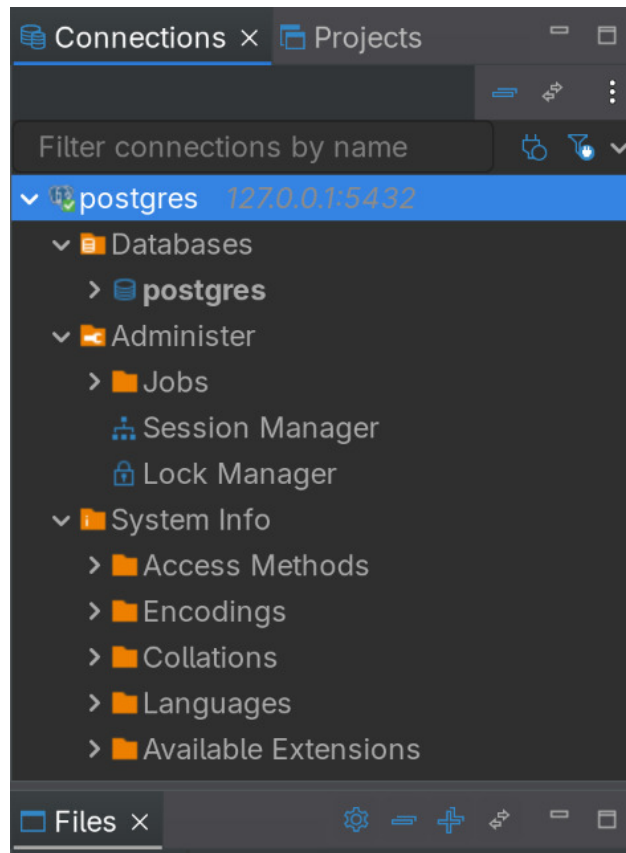


Figura 14: Conexión exitosa a la base de datos

Comentarios y problemas

Creo que la instalación en Arch Linux fue bastante sencilla. Aún así, tuve un inconveniente en la instalación: al iniciar el servicio de Docker manualmente, la terminal me lanzó el siguiente error:

```
Job for docker.service failed because start of the service was attempted too often.  
See "systemctl status docker.service" and "journalctl -xeu docker.service" for  
details.
```

```
To force a start use "systemctl reset-failed docker.service"
followed by "systemctl start docker.service" again.
```

Investigué un poco y este es un error que ocurre cuando el motor de Docker falla y activa su mecanismo de seguridad. En la *ArchWiki: Docker* [1] encontré varias recomendaciones, pero al seguir la más sencilla, que solo consistía en reiniciar mi computadora, funcionó a la perfección. Creo que el error se debió a no haber reiniciado mi computadora después de actualizar los paquetes del sistema, y no me dio más problemas tras hacerlo.

Por otro lado, investigué sobre cómo utilizar comandos de Docker sin utilizar `sudo` (agregando mi usuario al grupo `docker`). Sin embargo, encontré advertencias de múltiples usuarios sobre el riesgo de seguridad: el servicio de Docker se ejecuta con permisos de administrador, por lo que darle acceso a un usuario sin `sudo` equivale prácticamente a otorgarle privilegios administrativos completos sin contraseña (ya que un contenedor podría montar el directorio raíz del sistema anfitrión). Por las posibles brechas de seguridad que esto representa, decidí desertar de la idea y mejor consultar en clase cuáles son las mejores prácticas para manejar estos permisos.

Referencias

- [1] ArchWiki. (s. f.). *Docker*. <https://archlinux.org>